

The Hong Kong University of Science and Technology

COMP4321 Web Search Engines

Final Project Report

Project Title: **Web-based Search Engine with Spider, Indexer, and Retrieval**

Course	COMP4321 Web Search Engines
Semester	2026 Spring
Seed URL	https://hitcs1j.github.io/TestPages/testpage.htm
Indexed Pages	300
Team Name	[Fill Team Name]

Member 1	[Name, Student ID]
Member 2	[Name, Student ID]
Member 3	[Name, Student ID]

Submission Date: [Fill Date]

1 Overall Design of the System

1.1 System Overview

Our system contains four core modules:

1. **Spider/Crawler:** Crawls pages using BFS from a seed URL and extracts links.
2. **Indexer:** Performs stop-word removal, stemming, positional indexing, and metadata extraction.
3. **Retrieval Engine:** Executes vector-space ranking with cosine similarity and phrase matching.
4. **Web Interface (JSP):** Accepts queries and renders ranked results in required format.

1.2 Data Flow

1. Spider fetches page and extracts title/body/links.
2. Indexer transforms tokens (normalize $\rightarrow$ stop-word filter $\rightarrow$ stem).
3. Indexed data is persisted to JDBM structures.
4. Search engine reads indexes, computes scores, and returns top results.
5. JSP pages present results with score, metadata, keywords, and parent/child links.

This staged pipeline lets us validate each module independently during debugging and demo. For example, crawl correctness is observed through `spider_result.txt`, index correctness through DB inspection, and ranking correctness through query outputs.

2 File Structures Used in the Index Database

All persistent data is stored in one JDBM RecordManager (`searchengine`) using multiple named HTrees.

2.1 Core Mappings

- `url_to_pageID`: URL $\rightarrow$ pageID
- `pageID_to_url`: pageID $\rightarrow$ URL
- `word_to_wordID`: stem $\rightarrow$ wordID
- `wordID_to_word`: wordID $\rightarrow$ stem

2.2 Indexes and Metadata

- **bodyIndex**: wordID $\rightarrow$ postings (pageID, tf, positions)
- **titleIndex**: wordID $\rightarrow$ postings (pageID, tf, positions)
- **forwardIndex**: pageID $\rightarrow$ (wordID $\rightarrow$ tf)
- **pageMetadata**: pageID $\rightarrow$ title, URL, last-modified, size, maxTF

2.3 Link Graph and Counters

- **childLinks**: pageID $\rightarrow$ child pageIDs
- **parentLinks**: pageID $\rightarrow$ parent pageIDs
- **counters**: pageCounter, wordCounter

The schema combines both inverted and forward indexes. Inverted indexes are efficient for retrieval and ranking, while the forward index supports fast top-keyword extraction for result presentation. This dual design reduces query-time overhead and keeps UI rendering responsive.

3 Algorithms Used

3.1 Breadth-First Crawling

The crawler uses a queue-based BFS strategy and a visited set to avoid loops. URLs are mapped to IDs and link relations are stored bidirectionally (parent/child). Re-crawl skip logic is based on last-modified comparison. BFS was chosen for predictable traversal from the seed and stable behavior under crawl limits such as N=30 in hidden tests and N=300 for final indexing.

3.2 Token Processing

1. Lowercase and remove non-alphabetic characters.
2. Remove stop words from `stopwords.txt`.
3. Apply Porter stemming.
4. Store positions for phrase queries.

The same normalization and stemming are applied on query-side input, ensuring retrieval parity between indexed terms and query terms.

3.3 Ranking Model

For each term:

$$w_{d,t} = \frac{tf_{d,t}}{\max_d tf_d} \cdot idf_t, \quad idf_t = \log \left(\frac{N}{df_t} \right)$$

Query and document vectors are compared with cosine similarity:

$$\text{score}(d, q) = \frac{\sum_t w_{d,t} w_{q,t}}{\|d\| \|q\|}$$

In implementation, the query vector is built using the same idf values, and candidate documents are produced from posting lists of query terms/phrases. The final list is sorted in descending score order and truncated to top 50 as required by the specification.

3.4 Title Boosting

We add a bonus for title matches (term and phrase). This increases rank for pages where query intent appears in title text. The title boost can be toggled at runtime for demo: **SearchMain -no-title-boost ...** (CLI) and a checkbox in JSP UI. This allows direct before/after comparison using the same query. This design is easy to explain in demonstration: base relevance is first computed from body content, then title evidence is added to prioritize pages whose titles directly match user intent.

3.5 Phrase Search

Phrase matching is implemented by checking adjacency in positional postings for both title and body indexes. For a phrase of length k , if term 1 appears at position p , then term 2 must appear at $p + 1$, ..., term k at $p + k - 1$ in the same document field. This supports exact phrase lookup in both body and title. In practice, this removes false positives from non-adjacent token matches and improves precision for quoted queries.

3.6 Bonus Feature Implemented

Excluded terms/phrases:

- **-term**
- **-"exact phrase"**

Documents containing excluded conditions are filtered out before ranking output. This is integrated into query parsing and candidate filtering, so users can refine results without changing the base ranking model. Example: **university -science** removes pages that match **science**; **"hong kong" -"science park"** removes pages matching the excluded phrase while preserving other phrase matches. This bonus has low implementation risk but high demo value because users can immediately see result set changes with and without exclusion constraints.

4 Installation Procedure

4.1 Compile

```
javac -cp .:htmlparser.jar:lib/jdbm-1.0.jar \  
    IRUtilities/Porter.java StopStem.java Posting.java PageMetadata.java \  
    DBManager.java Crawler.java Spider.java QueryParser.java \  
    SearchEngine.java SearchMain.java TestProgram.java
```

4.2 Build 300-page DB

```
rm -f searchengine.db searchengine.lg
```

```
java -cp .:htmlparser.jar:lib/jdbm-1.0.jar Spider \  
  "https://hitcs1j.github.io/TestPages/testpage.htm" 300
```

4.3 Run CLI Retrieval

```
java -cp .:htmlparser.jar:lib/jdbm-1.0.jar SearchMain "computer science"
```

4.4 Run JSP UI on Tomcat

Deploy `web/search.jsp`, `web/results.jsp`, `web/WEB-INF/web.xml`, class files, and required jars to Tomcat webapp directory. Then open:

`http://localhost:8080/comp4321search/search.jsp`

5 Testing and Results

5.1 Functional Tests

- 300-page indexing from required seed URL.
- Hidden-seed simulation with $N=30$ from empty DB.
- Re-run skip behavior validation.
- Query tests: single term, multi-term, phrase, zero-result, stop-word-only.
- Stemming parity: `run` vs `running`, `book` vs `books`.
- Bonus tests for excluded terms and phrases.

Observed behavior during test runs confirms that hidden-seed $N=30$ crawling completes in demo-friendly time and immediate reruns index 0 additional pages when content is unchanged.

Insert terminal screenshot showing $N=300$ indexing completion and runtime.

Figure 1: Crawler output screenshot ($N=300$ indexing completion and runtime).

Insert terminal screenshot showing hidden-seed N=30 run and skip-on-rerun behavior.

Figure 2: Hidden-seed test screenshot (N=30 and rerun skip behavior).

```
bash: java: command not found...
(base) shaheer@fedora:~/Documents/COMP4321 Project$ java -cp ./htmlparser.jar:lib/jdbm-1.0.jar DBViewer 0

GLOBAL STATS
Total pages indexed : 317
Total unique stems : 14498

counters
pageCounter : 317
wordCounter : 14498

pageID_to_url / url_to_pageID
pageID URL
-----
0      https://hitcs1j.github.io/TestPages/testpage.htm
1      https://hitcs1j.github.io/TestPages/ust_cse.htm
2      https://hitcs1j.github.io/TestPages/news.htm
3      https://hitcs1j.github.io/TestPages/books.htm
4      https://hitcs1j.github.io/TestPages/Movie.htm
5      https://hitcs1j.github.io/TestPages/ust_cse/PG.htm
6      https://hitcs1j.github.io/TestPages/ust_cse/UG.htm
7      https://hitcs1j.github.io/TestPages/news/bbc.htm
8      https://hitcs1j.github.io/TestPages/news/cnn.htm
9      https://hitcs1j.github.io/TestPages/books/book1.htm
10     https://hitcs1j.github.io/TestPages/books/book2.htm

Ctrl+K to generate command
```

Figure 3: DB inspector output showing postings, term frequency, and positional data.

Search Results

☒ Title boost

[Back to search page](#)

Query: **computer**

Title boost: **ON**

Returned **7** result(s).

0.384771 **UG**

https://hitcslj.github.io/TestPages/ust_cse/UG.htm

Wed, 25 Mar 2026 13:03:26 HKT, 4157

program 16; comput 13; scienc 7; student 7; univers 6

Parent links

https://hitcslj.github.io/TestPages/ust_cse.htm

Child links

https://hitcslj.github.io/TestPages/ust_cse.htm

0.264761 **PG**

https://hitcslj.github.io/TestPages/ust_cse/PG.htm

Figure 4: Web UI result page showing score, title/URL hyperlinks, metadata, keywords, and parent/child links.

Search Results

☒ Title boost

[Back to search page](#)

Query: **cse department**

Title boost: **ON**

Returned **7** result(s).

7.513337 **CSE department of HKUST**

https://hitcslj.github.io/TestPages/ust_cse.htm

Wed, 25 Mar 2026 13:03:26 HKT, 412

admis 2; cse 2; depart 2; hkust 2; main 1

Parent links

<https://hitcslj.github.io/TestPages/testpage.htm>

https://hitcslj.github.io/TestPages/ust_cse/PG.htm

https://hitcslj.github.io/TestPages/ust_cse/UG.htm

Child links

https://hitcslj.github.io/TestPages/ust_cse/PG.htm

https://hitcslj.github.io/TestPages/ust_cse/UG.htm

<https://hitcslj.github.io/TestPages/testpage.htm>

Figure 5: Title boost enabled.

Search Results

☐ Title boost

[Back to search page](#)

Query: **cse department**

Title boost: **OFF**

Returned 7 result(s).

0.668798 **CSE department of HKUST**
https://hitcslj.github.io/TestPages/ust_cse.htm
Wed, 25 Mar 2026 13:03:26 HKT, 412
admis 2; cse 2; depart 2; hkust 2; main 1

Parent links
<https://hitcslj.github.io/TestPages/testpage.htm>
https://hitcslj.github.io/TestPages/ust_cse/PG.htm
https://hitcslj.github.io/TestPages/ust_cse/UG.htm

Child links
https://hitcslj.github.io/TestPages/ust_cse/PG.htm
https://hitcslj.github.io/TestPages/ust_cse/UG.htm
<https://hitcslj.github.io/TestPages/testpage.htm>

Figure 6: Title boost disabled for the same query; score gap changes.

Search Results

☐ Title boost

[Back to search page](#)

Query: **university**

Title boost: **OFF**

Returned 13 result(s).

0.148333 **UG**
https://hitcslj.github.io/TestPages/ust_cse/UG.htm
Wed, 25 Mar 2026 13:03:26 HKT, 4157
program 16; comput 13; scienc 7; student 7; univers 6

Parent links
https://hitcslj.github.io/TestPages/ust_cse.htm

Child links
https://hitcslj.github.io/TestPages/ust_cse.htm

0.084149 **Cannibal Women in the Avocado Jungle of Death (1988)**
<https://hitcslj.github.io/TestPages/Movie/146.html>

Figure 7: Bonus query example 1: excluded term filtering.

Search Results

☒ Title boost

[Back to search page](#)

Query: **university - science**

Title boost: **ON**

Returned **11** result(s).

0.084149 [Cannibal Women in the Avocado Jungle of Death \(1988\)](#)

<https://hitslj.github.io/TestPages/Movie/146.html>

Wed, 25 Mar 2026 13:03:26 HKT, 6918

avocado 12; movi 11; imdb 11; women 10; user 10

Parent links

<https://hitslj.github.io/TestPages/Movie.htm>

Child links

<https://hitslj.github.io/TestPages/Movie.htm>

0.051200 [Pitcher and the Pin-Up \(2004\)](#)

<https://hitslj.github.io/TestPages/Movie/40.html>

Figure 8: Bonus query example 2: excluded phrase filtering.

6 Conclusion

6.1 Strengths

- Complete end-to-end system from crawling to web retrieval.
- Proper phrase support through positional indexing.
- Clean separation of body/title indexes to support title emphasis.
- Practical bonus feature with clear demo value.

6.2 Weaknesses

- Single-threaded crawler can be improved for throughput.
- Limited query language compared with production search engines.
- Basic UI focused on correctness over visual polish.

6.3 Future Improvements

- Multi-threaded crawling with polite throttling.
- Better URL canonicalization and duplicate-content handling.
- Link-based ranking extension (e.g., PageRank).
- Enhanced UI interactions and result filtering controls.

Another practical enhancement is configurable DB path binding so CLI and servlet environments can share the same index location without manual file copying.

7 Contribution

Member	Contribution	Main Responsibilities
[Member A]	[%]	Spider/crawling and link graph logic
[Member B]	[%]	Indexing/database schema and retrieval implementation
[Member C]	[%]	Web interface, testing, documentation, and demo prep

Note: Ensure total contribution percentages sum to 100%.